



**亞洲大學**  
ASIA UNIVERSITY

---

# **Digital Image Processing**

## **Midterm Project Report**

# **Real-Time Face Detection and Expression Recognition**

**Student Name : Hanifi Abrar Setiawan**

**Student ID : 112021224**

**Instructor : DINH-TRUNG VU**

**2025-10-31**

# **Chapter 1 Introduction**

This project implements real-time face detection and expression recognition using computer vision techniques. The main objective is to detect facial features (eyes, nose, mouth), recognize facial expressions (smiling, surprised, squinting, neutral), and display real-time calculated ratios in a live video feed captured through a webcam. This project reinforces key concepts taught during Weeks 1-7 of the Digital Image Processing course including image acquisition, feature detection, geometric transformations, and real-time processing. Real-time display of facial landmarks and expression ratios helps visualize the mathematical foundations of facial analysis algorithms and demonstrates practical applications of computer vision in human-computer interaction systems.

## **Chapter 2 Background**

### **2.1 Face Detection and Landmark Recognition**

Face detection is a fundamental task in computer vision that involves locating human faces in digital images. Modern approaches utilize machine learning models like MediaPipe's face detection and face mesh solutions to identify facial features. The face mesh model provides 468 landmark points on each detected face, enabling precise localization of facial features like eyes, nose, mouth, and facial contours. These landmarks form the foundation for facial expression analysis by providing quantitative measurements of facial feature positions and shapes.

### **2.2 Eye Aspect Ratio (EAR) for Expression Recognition**

The Eye Aspect Ratio (EAR) is a widely-used metric in facial expression analysis, particularly for eye state detection. For each eye, it's calculated as the ratio of vertical eye landmark distances to horizontal eye landmark distances. A low EAR value indicates closed or squinted eyes, while a higher EAR value indicates open eyes. This metric is critical for detecting expressions like squinting and can be used to identify blink patterns and drowsiness detection applications.

### **2.3 Mouth Aspect Ratio and Expression Classification**

Facial expression recognition often relies on geometric relationships between facial landmarks. For smile detection, the mouth width (distance between corner landmarks) compared to the nose distance and mouth opening ratio (vertical opening to width) provide key indicators. A smiling expression typically shows increased mouth width with raised corners, while surprised expressions show increased vertical mouth opening relative to width. These quantitative measures enable automated expression classification algorithms.

## **Chapter 3 Methodology & Tools**

### **3.1 Workflow**

1. Capture live video from webcam using OpenCV VideoCapture.
2. Convert each frame to RGB format for MediaPipe processing.
3. Detect faces using MediaPipe Face Detection model.

4. Detect facial landmarks using MediaPipe Face Mesh model (468 landmark points).
5. Calculate geometric ratios for expression recognition (mouth width/height, eye EAR, smile indicators).
6. Classify facial expressions based on calculated ratios and thresholds.
7. Visualize results by drawing landmarks on facial features and displaying expression text/numerical values.
8. Display real-time metrics in fixed corner of video window for continuous monitoring.

### 3.2 Tools & Environment

Table 3.1. Tools & Environment

| Tools & Environment                             | Description                                    |
|-------------------------------------------------|------------------------------------------------|
| <b>Python 3.12</b>                              | Programming language                           |
| <b>Jupyter Lab</b>                              | A web-based IDE for Python                     |
| <b>OpenCV 4.12</b>                              | Image processing and visualization             |
| <b>Laptop Built-in Webcam</b>                   | Real-time video input                          |
| <b>MediaPipe 0.10</b>                           | Face detection and landmark recognition        |
| <b>Math Library</b>                             | Numerical calculations and geometric functions |
| <b>MediaPipe Face Detection &amp; Face Mesh</b> | Pre-trained models for facial analysis         |

## Chapter 4 Implementation

### 4.1 Early Stage: Face and Landmark Detection

The first step was to successfully capture frames from the webcam and implement face detection using MediaPipe's models. This confirmed that the environment was configured correctly and that both face detection and landmark recognition were working properly with real-time video streams. The MediaPipe models provided reliable face detection with good accuracy and performance for real-time applications.

Code snippet:

```
# Initialize MediaPipe Face Detection and Face Mesh solutions
self.mp_face_detection = mp.solutions.face_detection
self.mp_face_mesh = mp.solutions.face_mesh
self.mp_drawing = mp.solutions.drawing_utils
self.mp_drawing_styles = mp.solutions.drawing_styles

# Initialize face detection model
self.face_detection = self.mp_face_detection.FaceDetection(min_detection_confidence=0.5)

# Initialize face mesh model for landmarks
self.face_mesh = self.mp_face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=1,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)
```

Figure 4.1. Code snippet for Capturing Video Frame

Result:

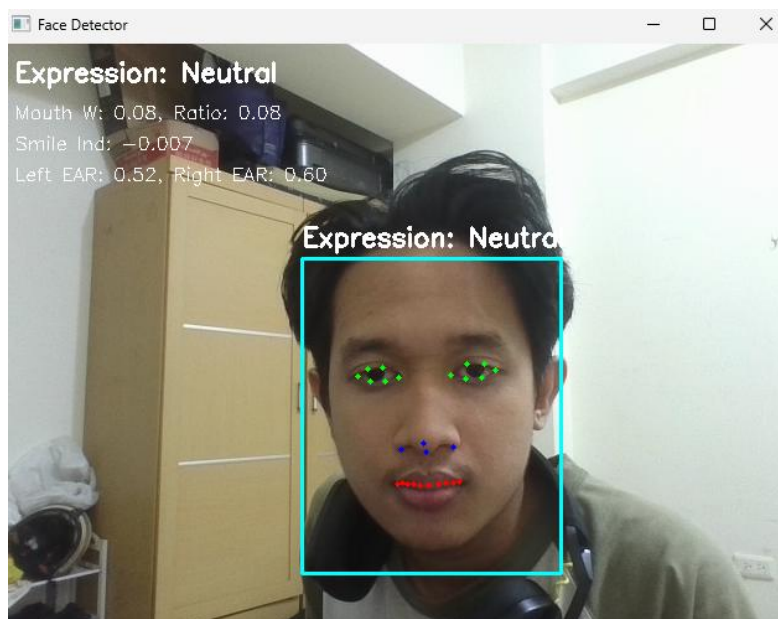


Figure 4.2 Face Detector initial display

## 4.2 Facial Feature Visualization

After successfully detecting facial landmarks, the next step was visualizing the key facial features by drawing colored dots on the landmarks. Eyes were marked in green, nose in blue, and mouth in red. This allowed verification that landmark detection was accurate and provided a clear visual representation of the facial features being analyzed.

**Code snippet:**

```
def draw_facial_features(self, image, landmarks):
    """Draw circles on facial features"""
    h, w, _ = image.shape

    # Draw eyes
    for idx in self.LEFT_EYE_INDICES:
        landmark = landmarks[idx]
        x, y = int(landmark.x * w), int(landmark.y * h)
        cv2.circle(image, (x, y), 2, (0, 255, 0), -1) # Green for eyes

    for idx in self.RIGHT_EYE_INDICES:
        landmark = landmarks[idx]
        x, y = int(landmark.x * w), int(landmark.y * h)
        cv2.circle(image, (x, y), 2, (0, 255, 0), -1) # Green for eyes

    # Draw nose
    for idx in self.NOSE_INDICES:
        landmark = landmarks[idx]
        x, y = int(landmark.x * w), int(landmark.y * h)
        cv2.circle(image, (x, y), 2, (255, 0, 0), -1) # Blue for nose

    # Draw mouth
    for idx in self.MOUTH_INDICES:
        landmark = landmarks[idx]
        x, y = int(landmark.x * w), int(landmark.y * h)
        cv2.circle(image, (x, y), 2, (0, 0, 255), -1) # Red for mouth
```

Figure 4.3. Code snippet for Facial Feature Visualization

## 4.3 Expression Recognition Algorithm Implementation

Expression recognition was implemented by calculating geometric relationships between facial landmarks. Key metrics included mouth width, mouth opening ratio, smile indicators (mouth corner height relative to mouth center), and eye aspect ratios (EAR) for each eye. These quantitative measures were then used to classify expressions based on threshold values determined through testing and refinement.

**Code Snippet :**

```
# Determine expression
if mouth_open_dist / mouth_width > 0.2:
    return "Surprised"
elif mouth_width > 0.13 and smile_indicator < -0.01: # More sensitive for smiling
    return "Smiling"
elif mouth_ratio > 0.10 and smile_indicator < -0.01: # Lower threshold for smile detection
    return "Smiling"
elif left_eye_ear < 0.50 or right_eye_ear < 0.50:
    return "Squinting"
else:
    return "Neutral"
```

Figure 4.4. Code snippet for Expression Recognition

**Result:**

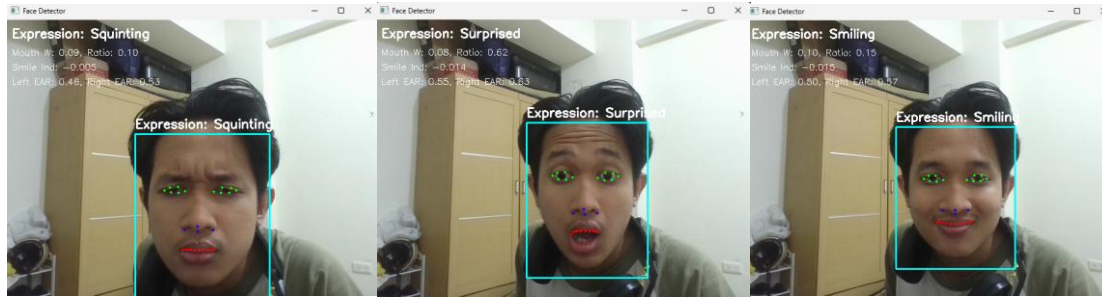


Figure 4.5 Expression Recognition

## 4.4 Real-Time Metrics Display and System Integration

Finally, all components were integrated into one real-time pipeline that displays facial landmarks, bounding boxes, expressions, and real-time calculated ratios. Metrics like mouth width, mouth ratio, smile indicator, and EAR values are continuously updated and displayed in a fixed corner of the video window, providing immediate feedback on the algorithm's analysis of facial features and expression classification.

**Code snippet:**

```
# Display ratio information in a fixed corner (top-left)
h, w, _ = image.shape
cv2.putText(image, f'Expression: {expression}',
            (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
cv2.putText(image, f'Mouth W: {mouth_width:.2f}, Ratio: {mouth_ratio:.2f}',
            (10, 60),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
cv2.putText(image, f'Smile Ind: {smile_indicator:.3f}',
            (10, 85),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
cv2.putText(image, f'Left EAR: {left_eye_ear:.2f}, Right EAR: {right_eye_ear:.2f}',
            (10, 110),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
```

Figure 4.6 Code Snippet for real-time Metrics

**Result:**



Figure 4.7 Metrics display

## **Chapter 5 Challenges and Solutions**

### **5.1 Challenge 1 - Expression Recognition Sensitivity**

#### **Description:**

Initial expression recognition thresholds were too strict, making it difficult to detect subtle expressions like gentle smiles or partial surprise. The algorithm was only recognizing very clear, exaggerated expressions, reducing its usefulness for natural facial expression detection.

#### **Cause:**

The threshold values for detecting expressions were set too high, requiring significant changes in facial feature geometry to trigger expression recognition. This resulted in many natural expressions being classified as "Neutral"

#### **Solution:**

Adjusted the sensitivity thresholds for expression detection by reducing the required mouth width change for smiles and mouth opening change for surprise. The thresholds were empirically tuned through testing with various facial expressions to achieve optimal sensitivity for natural expressions.

#### **Outcome:**

The algorithm now reliably detects more subtle expressions, making it more useful for real-world applications where people exhibit natural, less exaggerated facial expressions. The sensitivity for smile detection was improved by reducing mouth width thresholds from 0.18 to 0.13 and smile indicator thresholds from -0.02 to -0.01 for smile detection.

### **5.2 Challenge 2 – Real-time Display of Multiple Metrics**

#### **Description:**

Displaying all calculated metrics while maintaining clear visualization of facial features was challenging. The video display was becoming cluttered with text overlays making it difficult to focus on the face while still accessing the quantitative information.

#### **Solution:**

Implemented a fixed display area in the top-left corner of the video window that shows all calculated metrics continuously. This separates the facial landmark visualization from the metric display, allowing clear viewing of both facial features and quantitative analysis results without visual interference. Only the expression type is displayed near the face bounding box to keep the facial region clean while providing essential classification information.



**Outcome:**

Cleaner visualization with clear separation between facial landmark detection and quantitative metrics display, improving both the readability of metrics and the clarity of facial feature visualization simultaneously.

## **Chapter 6 Conclusion and Future Work**

### **6.1 Conclusion**

In this midterm project, I developed a real-time system that captures live video, detects faces and facial landmarks, recognizes facial expressions, and displays real-time quantitative metrics. Through this implementation, I gained practical experience in applying fundamental Digital Image Processing concepts such as image acquisition from live video streams, geometric transformations, feature detection, and real-time performance optimization.

I explored computer vision techniques including MediaPipe's pre-trained models for face detection and landmark recognition, and implemented geometric algorithms for expression classification based on facial landmark positions and relationships. This helped me understand the importance of geometric relationships in facial analysis and how quantitative measures can be derived from spatial relationships between facial features to create automated recognition systems.

The project also enhanced my understanding of real-time image processing challenges, particularly balancing computational efficiency with accuracy in facial analysis algorithms. By debugging and optimizing the system, I developed stronger programming skills in computer vision applications and improved my ability to translate theoretical concepts into functional visual analysis systems.

Overall, this project strengthened my foundation in computer vision and facial analysis techniques, which are key technologies for higher-level applications like emotion recognition, human-computer interaction, and advanced facial analysis systems.

### **6.2 Future Work**

For potential extensions of this work, several directions could be pursued to enhance the facial analysis capabilities. These include improving expression recognition accuracy by incorporating machine learning techniques to train custom expression classifiers, extending the system to detect additional facial features like eyebrows for more nuanced expression recognition, implementing facial recognition capabilities to identify specific individuals, and adding gesture recognition based on facial movements.

Additional improvements could include adding facial attribute detection like face shape classification, incorporating more sophisticated geometric analysis for complex expression recognition, and developing a more comprehensive facial analysis toolkit that could be used for emotion tracking, attention monitoring, or accessibility applications. This

foundation in facial landmark detection and expression analysis provides the groundwork for more advanced computer vision applications that require detailed understanding of facial features and expressions for human-centered computing applications.

## References and Appendix

### References:

1. OpenCV Documentation and Tutorials, retrieved from  
[https://docs.opencv.org/4.x/d6/d00/tutorial\\_py\\_root.html](https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html)
2. MediaPipe Face Detection and Face Mesh Documentation, Google AI, retrieved from  
[https://google.github.io/mediapipe/solutions/face\\_detection.html](https://google.github.io/mediapipe/solutions/face_detection.html)  
[https://google.github.io/mediapipe/solutions/face\\_mesh.html](https://google.github.io/mediapipe/solutions/face_mesh.html)

### Appendix:

- Source Code: *faceDetector.py*